

PocketSense/DhanChetna

Software Development Project Report

Contents

1	Executive Summary	2
2	Introduction	2
2.1	Purpose and Scope	2
2.2	Background and Motivation	2
2.3	Problem Statement	2
2.4	Project Objectives	3
3	Market Analysis	3
3.1	Existing Solutions	3
3.2	Gap Analysis	3
3.3	Competitive Advantage	3
4	Technical Approach	3
4.1	System Architecture	3
4.1.1	Architecture Pattern: Modular Monolithic Backend	4
4.1.2	Technology Stack	4
4.2	Database Design	4
4.2.1	Database Choice Rationale	4
4.2.2	Core Database Entities	4
4.3	API Design	5
4.3.1	RESTful API Principles	5
4.3.2	Authentication Flow	5
4.3.3	Key API Endpoints	5
4.4	Frontend Architecture	5
4.4.1	Web Application	5
4.4.2	Desktop Application	6
4.5	Intelligent Features Implementation	6
4.5.1	Context-Aware Profiling	6
4.5.2	Pattern Detection Algorithm	6
4.5.3	Advice Generation Engine	6
4.6	System Flow	7
5	Conclusion	7

1 Executive Summary

PocketSense (DhanChetna) is a context-aware expense tracking application specifically designed for college students living in hostels. The application addresses a critical gap in the current expense management market by focusing on the unique financial challenges faced by students with limited pocket money, where basic expenses (food and shelter) are already covered through hostel facilities.

Unlike existing expense tracking applications built for working professionals with complex financial portfolios, PocketSense provides simplified, context-aware financial management tailored to student life. The system identifies small, frequent spending patterns that accumulate unnoticed, provides intelligent advice based on the user's living situation, and helps develop long-term financial awareness.

The application is being developed using a modular architecture with Django REST Framework backend and multiple frontend options (web application and desktop application), allowing students to access their financial data across different platforms.

2 Introduction

2.1 Purpose and Scope

The purpose of this project is to develop a comprehensive expense tracking application that helps college students, particularly those living in hostels, understand and manage their limited pocket money more effectively. The scope includes:

- Transaction management (expense and income tracking)
- Context-aware user profiling based on living situation
- Budget setting and monitoring with real-time alerts
- Savings goal tracking with visual progress indicators
- Analytics and reporting features
- Pattern detection for repetitive small expenses
- Intelligent, personalized financial advice generation

2.2 Background and Motivation

During my time at a hostel in Bangalore for BCA studies, I experienced firsthand how pocket money seemed to disappear despite having food and accommodation covered. Analysis of my spending patterns revealed that 99.5% of monthly pocket money went to food purchases outside the mess, and barely 0.5% on travel. The problem wasn't lack of financial discipline—it was lack of awareness about cumulative small expenses.

This observation extends to most college students in hostels who face a common challenge: small purchases (Rs. 30 tea, Rs. 50 snacks, Rs. 100 printing) don't feel significant individually, but happen multiple times daily without conscious tracking. By mid-month, pocket money is exhausted without clear understanding of where it went.

2.3 Problem Statement

Current expense tracking applications are designed for working professionals with regular salaries, multiple income streams, investments, and complex financial portfolios. For students with simple pocket money, these applications are:

- Overly complex with irrelevant features like investment tracking, loan management, and credit score monitoring
- Not context-aware—they don't understand that hostel students with provided meals have fundamentally different spending patterns
- Generic in advice—they give the same budgeting recommendations to everyone without considering individual circumstances
- Bank-centric—many rely on SMS-based automatic tracking, useless for cash-based student transactions

2.4 Project Objectives

1. Develop a user-friendly expense tracking system tailored specifically for student lifestyles
2. Implement context-aware financial profiling that considers living situation and resource access
3. Create pattern detection algorithms to identify small, frequent spending behaviors
4. Build an intelligent advice engine that provides personalized recommendations based on user context
5. Design and deploy web and desktop frontends for cross-platform accessibility
6. Establish a scalable backend architecture that can support future enhancements

3 Market Analysis

3.1 Existing Solutions

Several expense tracking applications currently serve the Indian market:

Walnut: Automatically reads SMS alerts from banks to categorize spending. Offers budget limits, bill splitting, and credit card features. However, designed for users with active bank accounts and credit cards making regular digital transactions.

Moneyview: Syncs with bank accounts for automatic transaction tracking and provides features for personal loans, investments, and comprehensive financial management.

Money Manager by Realbyte: Popular free app allowing manual expense entry with categories, budgets, and reports. Includes loan calculators, interest calculators, and cloud backup features.

Monify: Focuses on quick expense entry with minimal interface, offering speed without automatic syncing or intelligent analysis.

YNAB (You Need A Budget): Comprehensive budgeting app requiring manual allocation of every rupee to specific categories. Powerful but steep learning curve, designed for stable, predictable incomes.

3.2 Gap Analysis

Common limitations across existing solutions:

- Focus on complex financial management (investments, loans, multiple accounts)
- Lack of context-aware interpretation of expenses
- Generic budgeting advice without personalization
- No specific focus on micro-expense pattern detection
- Overwhelming feature sets for users with simple financial needs

3.3 Competitive Advantage

PocketSense differentiates itself through:

- **Student-specific design:** Exclusively targeted at college students with simple income sources
- **Context awareness:** Understanding that hostel-provided meals changes how food spending should be interpreted
- **Micro-expense focus:** Specific detection of small, frequent purchases that accumulate unnoticed
- **Educational approach:** Emphasis on awareness and learning rather than restriction and control
- **Simplicity:** Deliberately excludes irrelevant features to reduce cognitive load

4 Technical Approach

4.1 System Architecture

The application follows a client-server architecture with clear separation of concerns:

4.1.1 Architecture Pattern: Modular Monolithic Backend

The system employs a modular monolithic architecture rather than microservices for the following reasons:

- **Development efficiency:** Single codebase reduces complexity for solo/small team development
- **Simplified deployment:** One artifact to deploy and manage
- **Easier debugging:** Unified error logs and stack traces
- **Cost effectiveness:** Single server instance reduces operational costs
- **Future scalability:** Modular structure allows extraction of specific components into microservices if bottlenecks emerge

4.1.2 Technology Stack

Backend Technologies:

- **Framework:** Django 5.2 with Django REST Framework for API development
- **Database:** PostgreSQL for production (ACID compliance for financial data), SQLite for development
- **Authentication:** Firebase Authentication for secure user management
- **Caching:** Redis for session management and performance optimization
- **Task Queue:** Celery for asynchronous processing (pattern detection, report generation)

Frontend Technologies:

- **Web Application:** Django templates with vanilla JavaScript and Chart.js for visualizations
- **Desktop Application:** Tkinter (Python standard library) for cross-platform desktop support
- **Future Mobile:** Progressive Web App (PWA) approach for mobile accessibility

Machine Learning Technologies:

- **Libraries:** scikit-learn for categorization and pattern recognition
- **NLP:** TF-IDF for expense title/description analysis
- **Time-series:** Prophet or ARIMA for spending prediction

4.2 Database Design

4.2.1 Database Choice Rationale

PostgreSQL was selected as the primary database for the following reasons:

- **ACID compliance:** Critical for financial transaction integrity
- **JSONB support:** Enables flexible storage for user preferences and metadata
- **Advanced features:** Window functions, full-text search, and complex queries
- **Django ORM compatibility:** First-class support in Django
- **Free hosting options:** Available on Neon, Supabase, and Render

4.2.2 Core Database Entities

User Management:

- **User:** Core authentication and identification
- **UserProfile:** Financial context (income type, living situation, resource access)

Transaction Management:

- **Transaction:** Individual expenses and income entries

- **Category:** System-default and user-custom expense categories
- **IncomeSource:** Registered income streams with frequency tracking

Financial Planning:

- **Budget:** Monthly spending limits per category
- **SavingsGoal:** Target-based savings with deposit tracking

Intelligence Layer:

- **RecurringPattern:** Detected repetitive spending behaviors
- **AuditLog:** Complete transaction history for accountability

4.3 API Design

4.3.1 RESTful API Principles

The backend exposes a RESTful API following these principles:

- **Resource-based URLs:** Endpoints represent resources (/transactions/, /budgets/)
- **HTTP methods:** GET (retrieve), POST (create), PUT (update), DELETE (remove)
- **Stateless communication:** Each request contains all necessary information
- **JSON format:** Standardized request/response format
- **Versioning:** API versioned at /api/v1/ for future compatibility

4.3.2 Authentication Flow

1. User authenticates via Firebase (email/password or Google Sign-In)
2. Firebase returns ID token (JWT)
3. Client includes token in Authorization header for all API requests
4. Django backend verifies token with Firebase on each request
5. Request proceeds with authenticated user context

4.3.3 Key API Endpoints

Authentication: POST /api/v1/auth/verify/

Transactions: GET/POST /api/v1/transactions/, GET/PUT/DELETE /api/v1/transactions/{id}/

Analytics: GET /api/v1/analytics/summary/, GET /api/v1/analytics/weekly/

Budgets: GET/POST /api/v1/budgets/, GET /api/v1/budgets/status/

Advice: GET /api/v1/advice/

4.4 Frontend Architecture

4.4.1 Web Application

The web frontend uses server-side rendering with Django templates for the following benefits:

- Fast initial page load
- SEO-friendly content
- Simpler deployment (single server serves both API and UI)
- Progressive enhancement with JavaScript

Key features:

- Responsive design for mobile and desktop browsers
- Chart.js for interactive data visualizations
- Dark/light theme toggle

4.4.2 Desktop Application

Built with Tkinter for cross-platform desktop support (Windows, macOS, Linux):

- Native feel on each operating system
- Local SQLite cache for offline functionality
- Automatic synchronization when online
- Consistent API client shared across platforms

4.5 Intelligent Features Implementation

4.5.1 Context-Aware Profiling

User profiles capture critical context:

- **Living situation:** Hostel, home, rented apartment, paying guest
- **Food provision:** Whether meals are included
- **Income type:** Pocket money, salary, stipend, freelance
- **Income frequency:** Weekly, bi-monthly, monthly

This context informs all subsequent analysis and advice generation.

4.5.2 Pattern Detection Algorithm

Phase 1 (Rule-Based):

1. Group transactions by normalized title and amount range
2. Detect frequency: same expense occurring 3+ times
3. Calculate interval standard deviation
4. Flag as recurring if interval consistency ≥ 2 days variance
5. Compute cumulative monthly impact

Phase 2 (ML-Enhanced):

1. TF-IDF vectorization of transaction descriptions
2. DBSCAN clustering to find similar-but-not-identical patterns
3. Time-series analysis for temporal patterns
4. Anomaly detection using Isolation Forest

4.5.3 Advice Generation Engine

The advice engine operates in layers:

Layer 1 - Rule-Based Advice (MVP):

- Amount thresholds: Classify expenses as minor ($\leq 10\%$ income), major ($> 10\%$)
- Frequency detection: Identify daily/weekly recurring expenses
- Context application: Interpret spending based on user profile

Example logic:

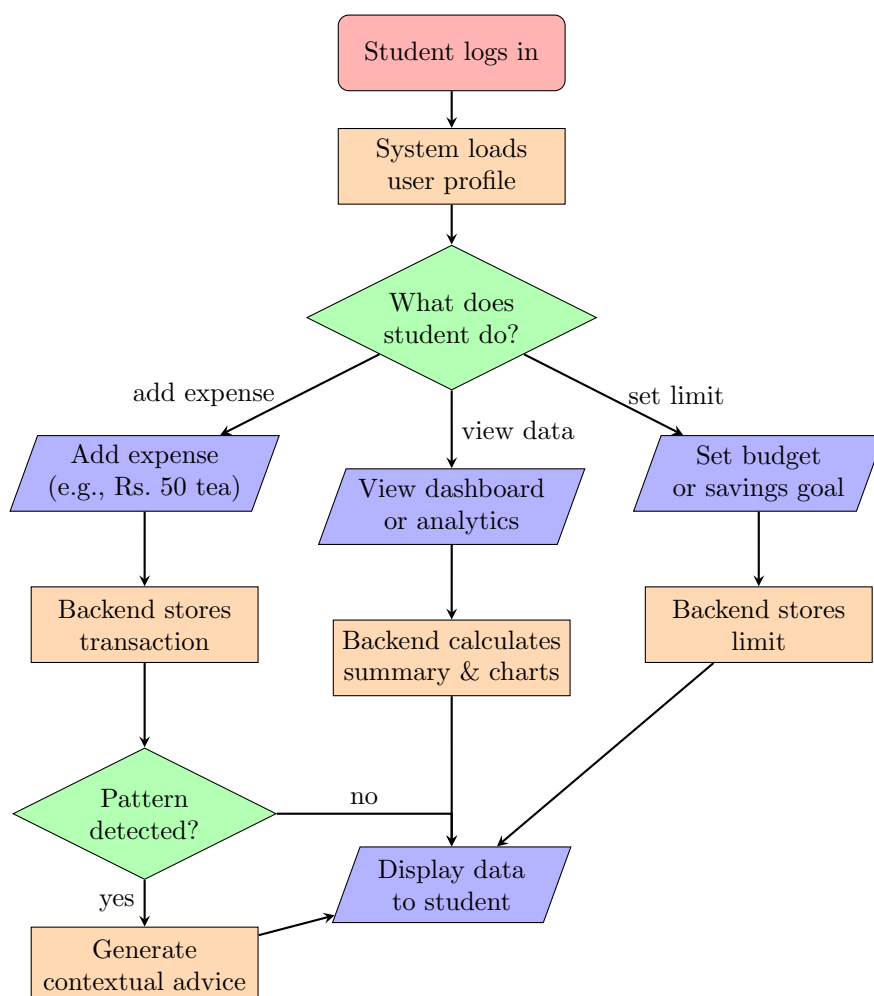
```
IF living_situation = "hostel"
  AND food_provided = true
  AND food_spending_ratio > 0.8
THEN advice = "Your food spending is unusually high
               for someone with provided meals.
               Consider whether this is compensating
               for meal quality."
```

Layer 2 - ML-Powered Advice (Future):

- Spending velocity prediction
- Budget exhaustion forecasting
- Peer comparison (anonymous, aggregated data)
- Personalized optimization suggestions

4.6 System Flow

The following diagram illustrates the complete user interaction flow:



5 Conclusion

PocketSense addresses a genuine problem faced by millions of college students in India—understanding where limited pocket money goes and developing better financial awareness. By focusing specifically on the student context rather than trying to be a universal finance app, it provides more relevant insights and advice than existing solutions.

The technical approach—a solid Django backend with multiple frontend options—demonstrates modern full-stack development skills while maintaining simplicity and cost-effectiveness. The phased development strategy allows for iterative improvement based on real user feedback.

Most importantly, the application takes an educational rather than restrictive approach. The goal is not to make students feel guilty about spending, but to help them understand their patterns and make informed choices. This philosophy of awareness over control is what distinguishes PocketSense from generic budgeting apps.

The project serves dual purposes: solving a real problem for students while providing comprehensive experience in backend development, API design, frontend frameworks, database modeling, machine learning integration, and deployment—all skills highly valued in the software industry.